C++ new/delete 用法详解（堆内存+内存泄漏避坑）

在C++中，`new` / `delete` 是专门用于**堆区内存手动管理**的操作符（关键字），用来替代C语言的 `malloc` / `free`，不仅能开辟/释放内存，还会自动调用**构造函数/析构函数**。

核心痛点：**栈内存自动释放，堆内存必须手动释放**，忘记释放就会造成**内存泄漏**，这是新手最常见的bug。

一、先搞懂：栈区 vs 堆区

这是理解 `new/delete` 的前提：

内存区域	开辟方式	释放方式	生命周期	适用场景
栈区	系统自动开辟	系统自动释放（函数结束即释放）	随函数/代码块结束	局部变量、临时数据
堆区	手动用 `new` 开辟	手动用 `delete` 释放	程序员完全控制	大数据、需要长期保存的数据

口诀：**栈自动管，堆手动管；new开堆，delete释堆。**

二、new/delete 核心用法

1. 单个对象：new / delete

语法

Code block

```
// 1. 开辟堆内存（无初始化）
类型* 指针名 = new 类型;
// 2. 开辟堆内存 + 初始化（推荐）
类型* 指针名 = new 类型(初始值);

// 释放堆内存
```

```
delete 指针名;
```

代码示例

Code block

```
#include <iostream>
using namespace std;

int main() {
    // 1. 堆区开辟int内存，初始值为10
    int* p = new int(10);
    cout << *p << endl; // 输出: 10

    // 2. 手动释放堆内存
    delete p;
    p = nullptr; // 【关键】置空，避免野指针

    return 0;
}
```

自定义类（核心优势：自动调用构造/析构）

`new` 会调用类的**构造函数**，`delete` 会调用**析构函数**：

Code block

```
class Test {
public:
    Test()  { cout << "构造函数执行" << endl; }
    ~Test() { cout << "析构函数执行" << endl; }
};

int main() {
    Test* p = new Test; // 堆上创建对象 → 调用构造
    delete p;           // 释放对象 → 调用析构 + 释放内存
    p = nullptr;
    return 0;
}
```

2. 数组对象：new[] / delete[]

⚠ **铁律**：数组必须用 `new[]` 开辟，`delete[]` 释放！**绝对不能混用！**

语法

Code block

```
// 开辟堆数组（n为数组长度）
类型* 指针名 = new 类型[n];
// 带初始化的堆数组
类型* 指针名 = new 类型[n]{值1, 值2, ...};

// 释放堆数组
delete[] 指针名;
```

代码示例

Code block

```
int main() {
    // 堆上开辟长度为5的int数组
    int* arr = new int[5]{1,2,3,4,5};
    cout << arr[2] << endl; // 输出: 3

    // 【必须用delete[]】释放数组
    delete[] arr;
    arr = nullptr;

    return 0;
}
```

❌ **错误示范**（必踩坑）：

Code block

```
int* arr = new int[5];
delete arr; // 错误! 只用delete → 只释放第一个元素，剩余内存泄漏!
```

三、什么是内存泄漏?

定义

用 `new` 在**堆区申请了内存，但没有用** `delete` **释放**，这块内存会被程序一直占用，直到程序结束，系统才会回收。

长期运行的程序（服务器、客户端）会因内存泄漏**耗尽系统内存，最终崩溃**。

常见泄漏场景

1. 忘记写 `delete`

Code block

```
int* p = new int(10); // 开辟内存
// 没有delete → 内存泄漏!
```

2. 指针丢失堆内存地址

Code block

```
int* p = new int(10);
p = new int(20); // 原内存地址被覆盖，永远丢失 → 泄漏!
```

3. `new[]` 和 `delete` 混用

Code block

```
int* arr = new int[5];
delete arr; // 数组未完全释放 → 泄漏!
```

四、5条铁律：彻底避免内存泄漏

1. 严格配对使用

- `new` ↔ `delete`
- `new[]` ↔ `delete[]`

不配对 = 内存泄漏 + 析构异常

2. delete 后必须置空 `nullptr`

`delete` 只会释放内存，不会清空指针，指针会变成**野指针**（指向无效内存），置空后更安全：

Code block

```
delete p;
p = nullptr; // 必加!
```

3. 不让指针丢失堆地址

不要随意给指向堆内存的指针重新赋值。

4. 不用则立即释放

堆内存用完后，**第一时间** `delete`，不要等到程序结束。

5. 【C++11 最佳实践】用智能指针（根治泄漏）

C++11 提供 `std::unique_ptr` / `std::shared_ptr` **智能指针**，会**自动调用delete**，完全不用手动管理内存，从根源杜绝泄漏：

Code block

```
#include <iostream>
#include <memory> // 智能指针头文件
using namespace std;

int main() {
    // 智能指针：自动释放堆内存，无需delete!
    unique_ptr<int> p(new int(10));
    cout << *p << endl;

    // 离开作用域时，自动释放内存 → 无泄漏
    return 0;
}
```

五、补充知识点

1. **delete 空指针是安全的**

直接 `delete nullptr;` 不会报错，无需判断。

2. **new 失败的处理**

默认 `new` 失败会抛异常 `std::bad_alloc`，也可以用 `nothrow` 让失败返回空指针：

Code block

```
int* p = new(nothrow) int; // 失败返回nullptr，不抛异常
```

总结

1. **`new` 开辟堆内存**，`delete` 释放堆内存；数组用 `new[]` / `delete[]`；

2. 内存泄漏 = 堆内存申请后**未释放**，长期运行会导致程序崩溃；
3. 避坑核心：**配对使用 + delete后置空 + 优先用智能指针**。

（注：文档部分内容可能由 AI 生成）